

My Project

Generated by Doxygen 1.14.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Student Class Reference	7
4.1.1 Member Function Documentation	8
4.1.1.1 gautiPavarde()	8
4.1.1.2 gautiVarda()	8
4.2 Zmogus Class Reference	8
5 File Documentation	9
5.1 failuapdorojimas.h	9
5.2 student.h	10
5.3 zmogus.h	10
Index	11

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	8
Student	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Student		7
Zmogus		8

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

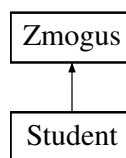
failuapdorojimas.h	9
student.h	10
zmogus.h	10

Chapter 4

Class Documentation

4.1 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- **Student** (std::string vardas, std::string pavarde, std::vector< int > nd, int egz)
- **Student** (const [Student](#) &other)
- **Student** ([Student](#) &&other) noexcept
- [Student](#) & **operator=** (const [Student](#) &other)
- [Student](#) & **operator=** ([Student](#) &&other) noexcept
- void **apskaiciuotiGalutinius** ()
- const std::string & **gautiVarda** () const override
- const std::string & **gautiPavarde** () const override
- const std::vector< int > & **gautiNd** () const
- int **gautiEgzamina** () const
- double **gautiVidurki** () const
- double **gautiMediana** () const

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (std::string vardas, std::string pavarde)

Friends

- std::ostream & **operator<<** (std::ostream &os, const [Student](#) &s)
- std::istream & **operator>>** (std::istream &is, [Student](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- `std::string vardas_`
- `std::string pavarde_`

4.1.1 Member Function Documentation

4.1.1.1 `gautiPavarde()`

```
const std::string & Student::gautiPavarde () const [override], [virtual]
```

Implements [Zmogus](#).

4.1.1.2 `gautiVarda()`

```
const std::string & Student::gautiVarda () const [override], [virtual]
```

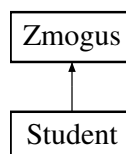
Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- `student.h`
- `student.cpp`

4.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (`std::string vardas, std::string pavarde`)
- `virtual const std::string & gautiVarda () const =0`
- `virtual const std::string & gautiPavarde () const =0`

Protected Attributes

- `std::string vardas_`
- `std::string pavarde_`

The documentation for this class was generated from the following files:

- `zmogus.h`
- `zmogus.cpp`

Chapter 5

File Documentation

5.1 failuapdorojimas.h

```
00001 #ifndef FAILUAPDOROJIMAS_H
00002 #define FAILUAPDOROJIMAS_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <iterator>
00007 #include <type_traits>
00008 #include <utility>
00009 #include <algorithm>
00010 #include "student.h"
00011
00012 std::vector<Student> nuskaitytiIsFailo(const std::string& filename);
00013 void rikiuotiPagalBalus(std::vector<Student>& studentai);
00014 void skirstytiStudentus(const std::vector<Student>& studentai,
00015                        std::vector<Student>& kietekai,
00016                        std::vector<Student>& vargsiukai);
00017 void irasytiIFaila(const std::vector<Student>& studentai, const std::string& filename);
00018
00019 template<typename Container>
00020 void splitStudentsCopyStrategy(const Container& students, Container& kietekai, Container& vargsiukai)
00021 {
00022     for (const auto& s : students) {
00023         if (s.gautiVidurki() >= 5.0)
00024             kietekai.push_back(s);
00025         else
00026             vargsiukai.push_back(s);
00027     }
00028 }
00029
00030 template<typename Container>
00031 void splitStudentsMoveStrategy(Container& students, Container& vargsiukai) {
00032     auto it = students.begin();
00033     while (it != students.end()) {
00034         if (it->gautiVidurki() < 5.0) {
00035             vargsiukai.push_back(std::move(*it));
00036             it = students.erase(it);
00037         } else {
00038             ++it;
00039         }
00040     }
00041 }
00042
00043 template<typename Container>
00044 void splitStudentsPartitionStrategy(Container& students, Container& kietekai, Container& vargsiukai) {
00045     typedef typename std::iterator_traits<typename Container::iterator>::iterator_category category;
00046     if (std::is_same<category, std::random_access_iterator_tag>::value) {
00047         auto partitionPoint = std::stable_partition(students.begin(), students.end(), [](const
00048 Student& s) {
00049     return s.gautiVidurki() >= 5.0;
00050 });
00051         kietekai = Container(students.begin(), partitionPoint);
00052         vargsiukai = Container(partitionPoint, students.end());
00053     } else {
00054         splitStudentsCopyStrategy(students, kietekai, vargsiukai);
00055     }
00056 }
00057
00058 #endif // FAILUAPDOROJIMAS_H
```

5.2 student.h

```

00001 #ifndef STUDENT_H
00002 #define STUDENT_H
00003
00004 #include "zmogus.h"
00005 #include <vector>
00006 #include <iostream>
00007
00008 class Student : public Zmogus {
00009 private:
00010     std::vector<int> nd_;
00011     int egzaminas_;
00012     double galutinisVidurkis_;
00013     double galutineMediana_;
00014 public:
00015     // Konstruktoriai
00016     Student();
00017     Student(std::string vardas, std::string pavarde, std::vector<int> nd, int egz);
00018
00019     // Rule of Five
00020     ~Student();
00021     Student(const Student& other);
00022     Student(Student&& other) noexcept;
00023     Student& operator=(const Student& other);
00024     Student& operator=(Student&& other) noexcept;
00025
00026     // Kiti metodai
00027     void apskaiciuotiGalutinius();
00028
00029     const std::string& gautiVarda() const override;
00030     const std::string& gautiPavarde() const override;
00031     const std::vector<int>& gautiNd() const;
00032     int gautiEgzamina() const;
00033     double gautiVidurki() const;
00034     double gautiMediana() const;
00035
00036     // Operatoriai
00037     friend std::ostream& operator<<(std::ostream& os, const Student& s);
00038     friend std::istream& operator>>(std::istream& is, Student& s);
00039 };
00040
00041 double skaiciuotiVidurki(const std::vector<int>& paz);
00042 double skaiciuotiMediana(std::vector<int> paz);
00043
00044 #endif // STUDENT_H
00045

```

5.3 zmogus.h

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 class Zmogus {
00007 protected:
00008     std::string vardas_;
00009     std::string pavarde_;
00010 public:
00011     Zmogus(std::string vardas, std::string pavarde) : vardas_(std::move(vardas)),
00012     pavarde_(std::move(pavarde)) {}
00013     virtual ~Zmogus() = 0;
00014     virtual const std::string& gautiVarda() const = 0;
00015     virtual const std::string& gautiPavarde() const = 0;
00016 };
00017
00018 #endif // ZMOGUS_H
00019

```

Index

gautiPavarde
 Student, [8](#)
gautiVarda
 Student, [8](#)

Student, [7](#)
 gautiPavarde, [8](#)
 gautiVarda, [8](#)

Zmogus, [8](#)